

Procesos

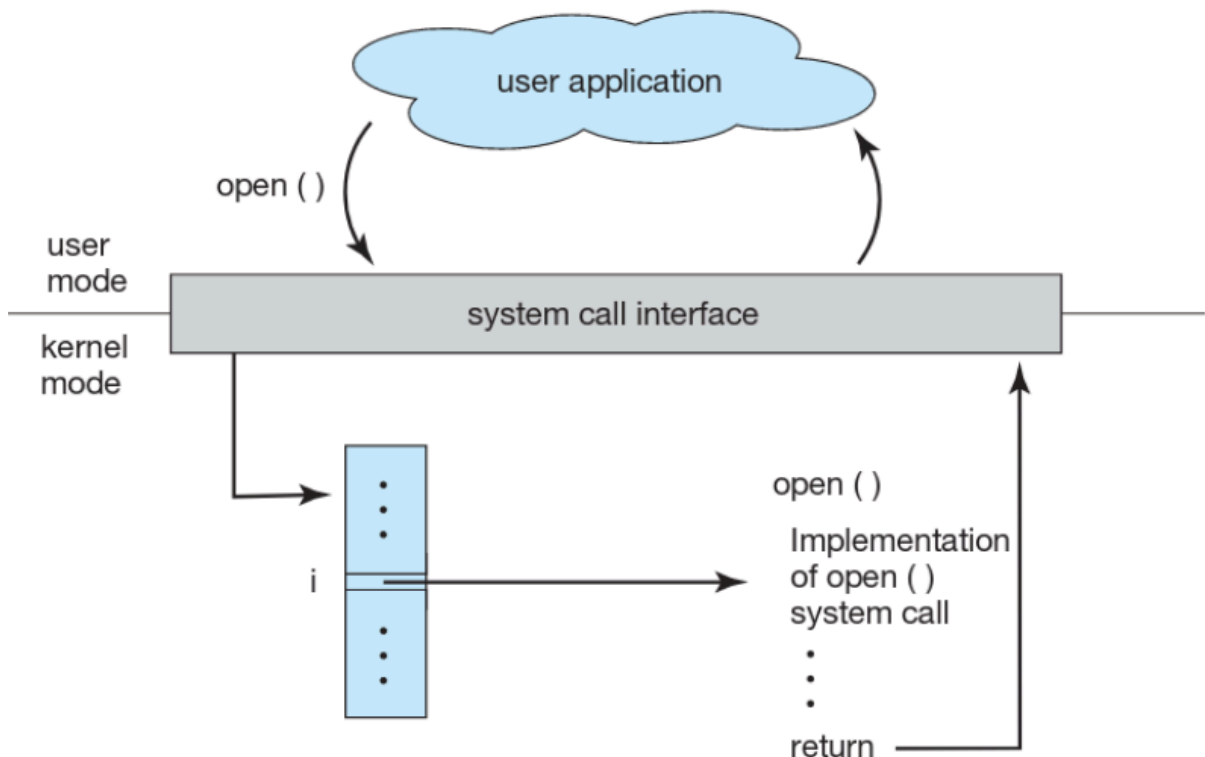
¿Qué es una system call? ¿Para qué se usan? ¿Cómo funcionan? Explicar en detalle el funcionamiento una system call en particular.

Son una interfaz para los servicios que brinda el sistema operativo. Generalmente son funciones escritas en C y C++. Es la API del sistema operativo.

Se usan para que un programa acceda al hardware, manipule archivos y/o operaciones de input/output. Es una forma estandarizada para que los programas interactuen con el sistema operativo y usen recursos del sistema de forma segura.

Suponiendo que un programa de usuario llama a una syscall:

1. Se settean los argumentos.
2. Se ejecuta la instrucción "syscall" para entrar a modo kernel.
3. En modo kernel se busca la syscall en la syscall table que quiere el usuario.
4. Se ejecuta el código en modo kernel de la syscall.
5. Se retorna a usuario.



¿Las system calls son universales a todos los sistemas operativos o son dependientes de cada sistema?

No necesariamente. Cada sistema operativo tiene su conjunto de syscalls.

Existe el estandar POSIX (IEEE 1003) para que los programas puedan ser compatibles en otros sistemas operativos, de esta manera si yo llamo "open()" en mi programa en el sistema operativo X el comportamiento va a ser similar a usar "open()" en mi programa en el sistema operativo Y.

¿Para qué sirve la system call fork? ¿Qué debilidades tiene? Comparar con vfork y la creación de threads.

Sirve para crear un proceso identico al actual que lo llamó. El proceso creado se llama "child" y el proceso que lo creó se llama "parent". Ambos procesos corren en espacios de memoria diferentes pero identicos al momento de forkear.

Se podría considerar una debilidad que el proceso hijo no puede acceder directamente al espacio de memoria original.

"vfork()" no copia el espacio de memoria del proceso "parent". Se usa porque tiene menos overhead (todo el copiado de paginas) si simplemente quieres hacer un execv.

exec reemplaza toda la imagen del proceso hijo por lo que no tiene sentido copiar la imagen del padre

Diferencias entre system calls para crear procesos entre Linux y Windows.

	Windows	Unix
Process	CreateProcess()	fork()
Control	ExitProcess()	exit()
	WaitForSingleObject()	wait()

¿Cómo funcionan los estados de un proceso? Ready, bloqueado, running. Explicar las transiciones de cada estado a cada estado (en particular, de waiting a ready).

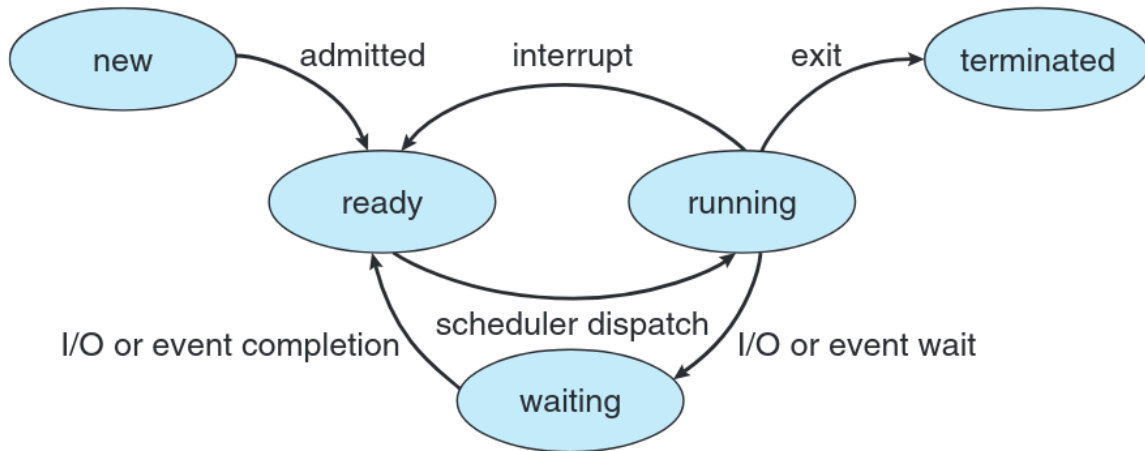
New: El proceso esta siendo creado.

Ready: El proceso esta en memoria principal esperando para que se le asigne un procesador.

Running: Las instrucciones del codigo estan siendo ejecutadas.

Waiting: El proceso esta esperando a que ocurra un evento como esperando input/output o alguna señal.

Terminated: El proceso termino su ejecucion.



New -> Ready: El proceso se creo con exito y esta listo para ejecutarse.

Ready -> Running: El scheduler le asigno un procesador al proceso.

Running -> Ready: El scheduler desalojo el proceso del cpu por alguna politica de este. (por ejemplo se termino su quanta o hay otro proceso con mas prioridad).

Preemption, se interrumpe temporalmente con la intencion de volver a su ejecucion mas tarde.

Running -> Waiting: El proceso envia una syscall avisando que necesita esperar I/O o que quiere usar un mutex. El kernel pone el proceso en waiting hasta que suceda algo relacionado.

Waiting -> Ready: Se genera un evento donde el proceso no tiene que esperar mas, el kernel mueve al proceso del estado waiting al estado ready entrando nuevamente al pool de procesos esperando un cpu para continuar su ejecucion. Esto ultimo lo decide el scheduler.

Running -> Terminated: El proceso termino su ejecucion.

¿Qué estructura se debe mantener en memoria para poder tener procesos?
Hablar de la tabla de procesos.

Se debe mantener una tabla de procesos donde cada entrada es una PCB (process control block). Estas estructuras mantienen el estado de los procesos actualmente activos en el sistema.

Cada PCB tiene:

Process state: El estado actual del proceso que vimos arriba en el diagrama.

(Running, ready, waiting, etc).

Process number: El process id.

Program counter: Siguiente instruccion a ejecutar.

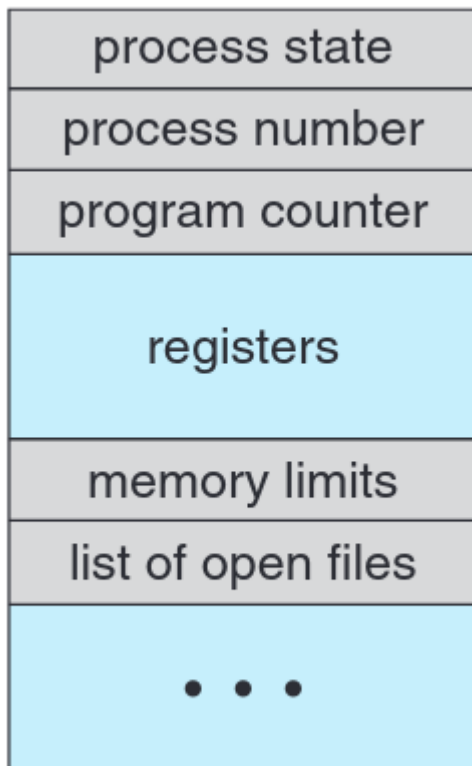
Registros: El estado de los registros al momento de interrumpir el proceso. Sirve para restaurar los valores cuando volvamos a ejecutarlo. Es como un snapshot y al momento de restaurar el proceso todo siga como si no hubiera pasado nada.

CPU Scheduling info: Proridad y parametros.

Memory Managment: Page tables, segment tables, base and limit registers.

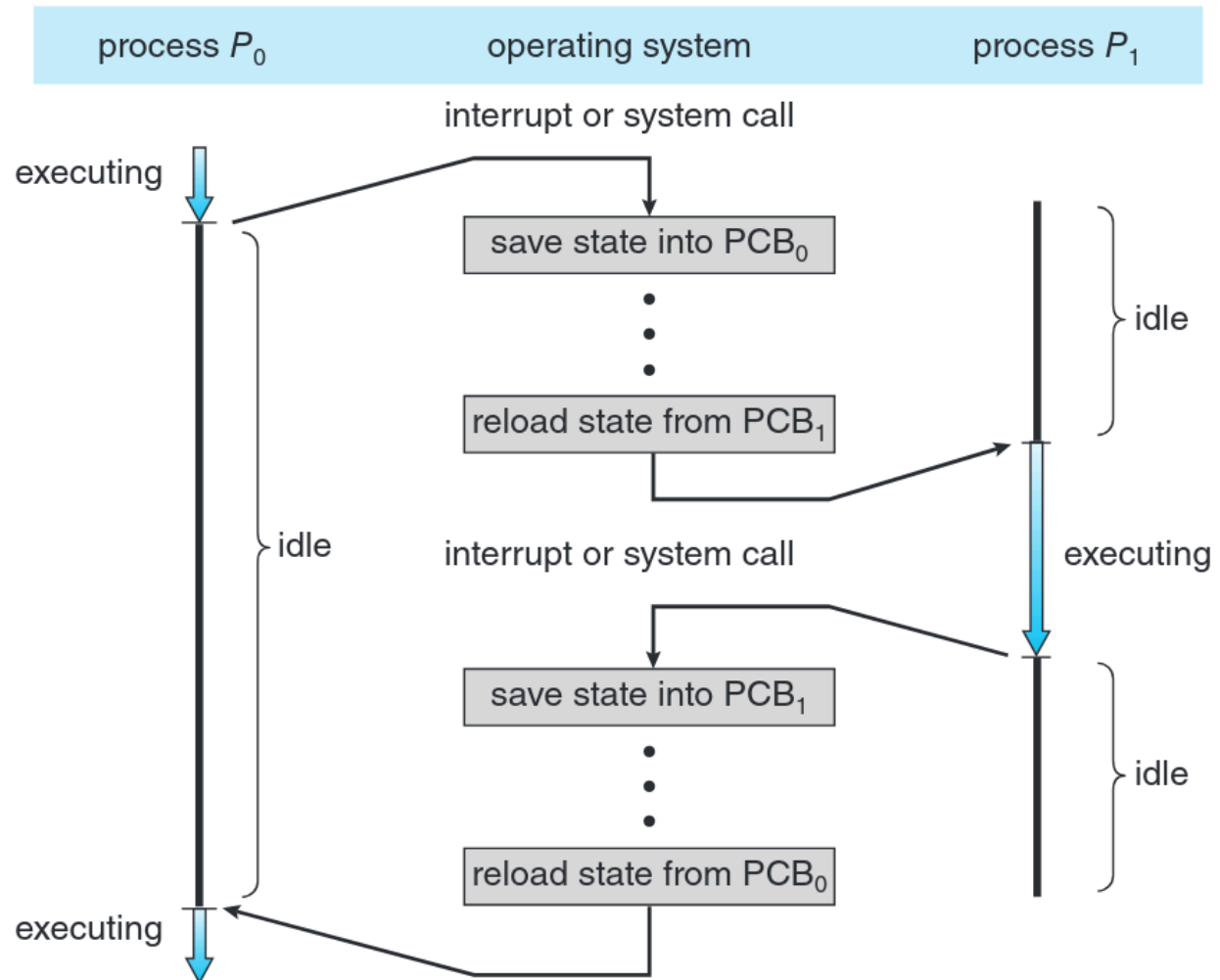
Accounting information: CPU usage, time limits, account numbers, job or process number.

I/O Status data: Lista de dispositivos de entrada o salida dedicadas al proceso, lista de archivos abiertos, etc.



Para swappear los procesos que corren en un CPU se hace un *context switch* donde se guardan los datos del pcb del proceso corriendo actualmente y se cargan los datos del

pcb del nuevo proceso a ejecutar. Este proceso tiene mucho *overhead*.



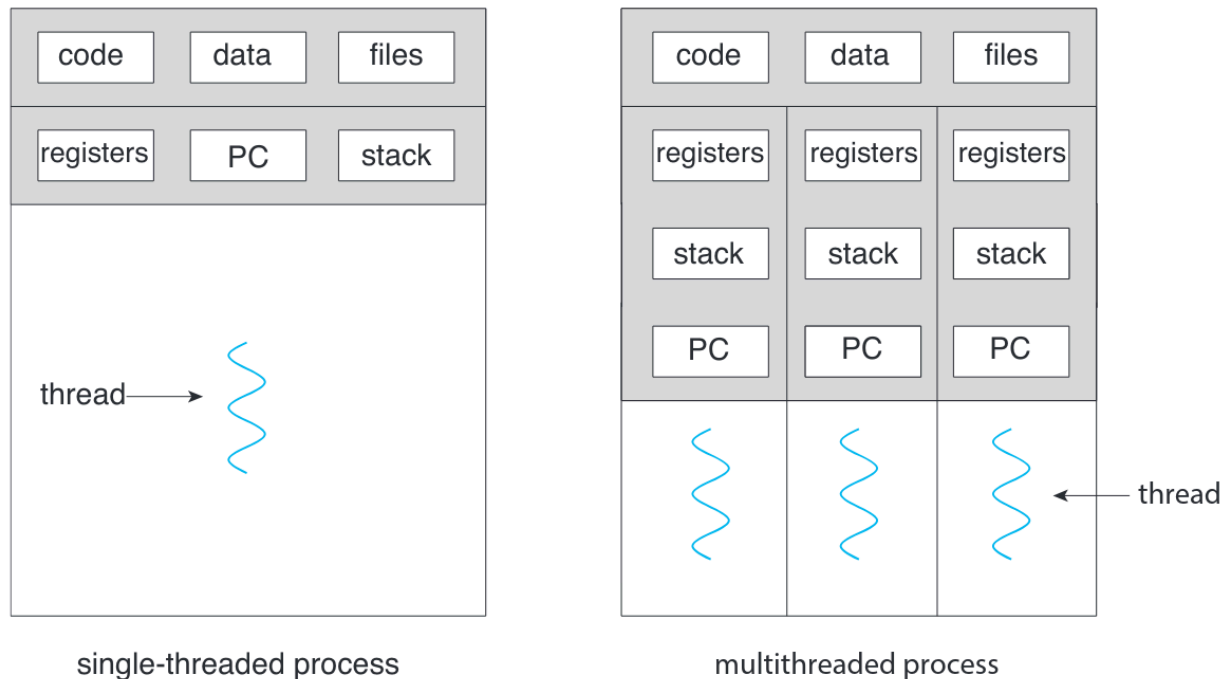
¿Qué es un proceso, un thread y en qué se diferencian?

Un *proceso* es la instancia dinamica de un programa corriendo en una computadora en una seccion de la memoria intensivo en el uso de recursos. Representa la ejecucion de un programa y es administrado por el sistema operativo.

Un *thread* es una pequena unidad de ejecucion **dentro** de un proceso.

La diferencia es que dos procesos son independientes aun si son la ejecucion del mismo programa. Tienen espacio de memoria diferentes, tienen PCBs diferentes y estados diferentes. Los threads comparten el mismo espacio de memoria dentro de un proceso y recursos aunque tienen un stack y registros diferentes. (comparten las mismas variables globales).

Los procesos se usan para *correr aplicaciones de manera independiente* mientras que los threads se usan para tareas paralelas dentro de un proceso.



¿Qué debería agregar a la PCB para manejar los threads?

Se expande el PCB para incluir información de cada thread (pág. 110).

Se deben incluir una sección para cada thread dedicado a registros, stack de ejecución y program counter. No es necesaria la sección de memoria porque los threads la comparten.

¿Qué pasaría si los threads compartieran el stack?

Comportamiento inesperado. Supongamos que dos threads T1 y T2 se separaron y están corriendo en funciones diferentes. Ambos están por terminar de ejecutar una función, T1 está por retornar (return address en el top del stack) pero justo se hace un context switch y T2 ahora será el que va a retornar. T2 podría retornar a la función llamadora de la función donde estaba T1 implicando que saltó a otra sección de texto que no le correspondía. Mientras, al volver T1 a ejecutar, perdió su dirección de retorno y va a "retornar" a un valor random.

¿Qué tendría que ver en un sistema para que piense que va a andar mejor agregando:

más procesadores.
más memoria.

1. Más procesadores sería sensato si hay demasiados procesos en ready debido a la falta de recursos del CPU.

2. Si se abusa del swap (memoria ram a disco y viceversa) o proceso bloqueados porque esperan memoria generando cuellos de botella. Tambien si hay muchas page faults.

Hablar de strace y ptrace.

strace es un logger de un proceso en un alto nivel mientras que ptrace es un debugger que puede manipular el proceso.

Strace es una herramienta de logs que ayuda al usuario a ver como interactua un proceso con el kernel. Monitorea system calls, senales, argumentos de retorno, etc.

Ptrace es una system call que permite a un proceso observar y controlar la ejecucion de otro proceso. Es usado por ejemplo en *gdb* para:

1. Pausar un proceso.
2. Inspeccionar y modificar sus registros o memoria.
3. Interceptar y modificar syscalls.
4. Administrar senales enviadas al proceso objetivo.